Министерство образования Республики Беларусь

Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Программирование мобильных приложений»

ОТЧЁТ
к лабораторной работе №2
на тему
«ТАЙМЕР»
БГУИР 6-05-0612-02 23

Выполнил студент группы 353503
СЕБЕЛЕВ Дмитрий Юрьевич

_____
(дата, подпись студента)

Проверил ассистент каф. информатики
БУРАКОВ Дмитрий Игоревич

_____
(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1 Постановка задачи 3
2 Выполнение работы 4
2.1 Архитектура и состав приложения 4
2.2 Экран списка последовательностей 5
2.3 Экран таймера 7
2.4 Экран настроек 8
2.5 Уведомление таймера 9
Вывод 11
Приложение А (обязательное) Листинг программного кода 12

1 ПОСТАНОВКА ЗАДАЧИ

Целью лабораторной работы являлась разработка мобильного приложения «Таймер» для операционной системы *Android*. Приложение должно обеспечивать управление многофазными интервальными таймерами с поддержкой создания, редактирования, удаления и воспроизведения пользовательских последовательностей.

Функциональные требования к приложению включают следующие возможности:

– Создание последовательностей таймеров с произвольным количеством фаз, где каждая фаза характеризуется типом активности, длительностью и количеством повторений.

– Сохранение созданных последовательностей между запусками приложения с использованием локального хранилища данных или механизмов сериализации.

– Отображение списка всех созданных последовательностей с возможностью выбора, редактирования и удаления элементов.

– Воспроизведение выбранной последовательности с визуализацией текущей фазы, оставшегося времени и индикатором прогресса.

– Управление воспроизведением таймера: запуск, пауза, остановка, переход к предыдущей и следующей фазам.

– Воспроизведение звукового сигнала при завершении каждой фазы для информирования пользователя о переходе.

– Работа таймера в фоновом режиме при свёрнутом приложении с отображением уведомлений и возможностью управления из системной панели.

– Настройка параметров интерфейса: выбор темы оформления (светлая или тёмная), изменение размера шрифта, выбор языка интерфейса.

– Управление данными приложения: удаление всех последовательностей и сброс настроек к значениям по умолчанию.

– Отображение экрана-заставки при запуске приложения.

2 ВЫПОЛНЕНИЕ РАБОТЫ

2.1 Архитектура и состав приложения

Разработанное приложение построено с использованием чёткого разделения уровня представления, бизнес-логики и данных. Уровень представления реализован с использованием декларативного подхода и включает экраны списка последовательностей, редактирования, воспроизведения таймера и настроек. Для обеспечения работы таймера в фоновом режиме реализован специализированный сервис.

Основные взаимодействия пользователя с приложением представлены в диаграмме вариантов использования на рисунке 2.1, а логика выполнения последовательности таймера отражена в диаграмме активностей на рисунке 2.2. Детальное описание экранов приложения и фонового сервиса приведено в последующих подразделах.

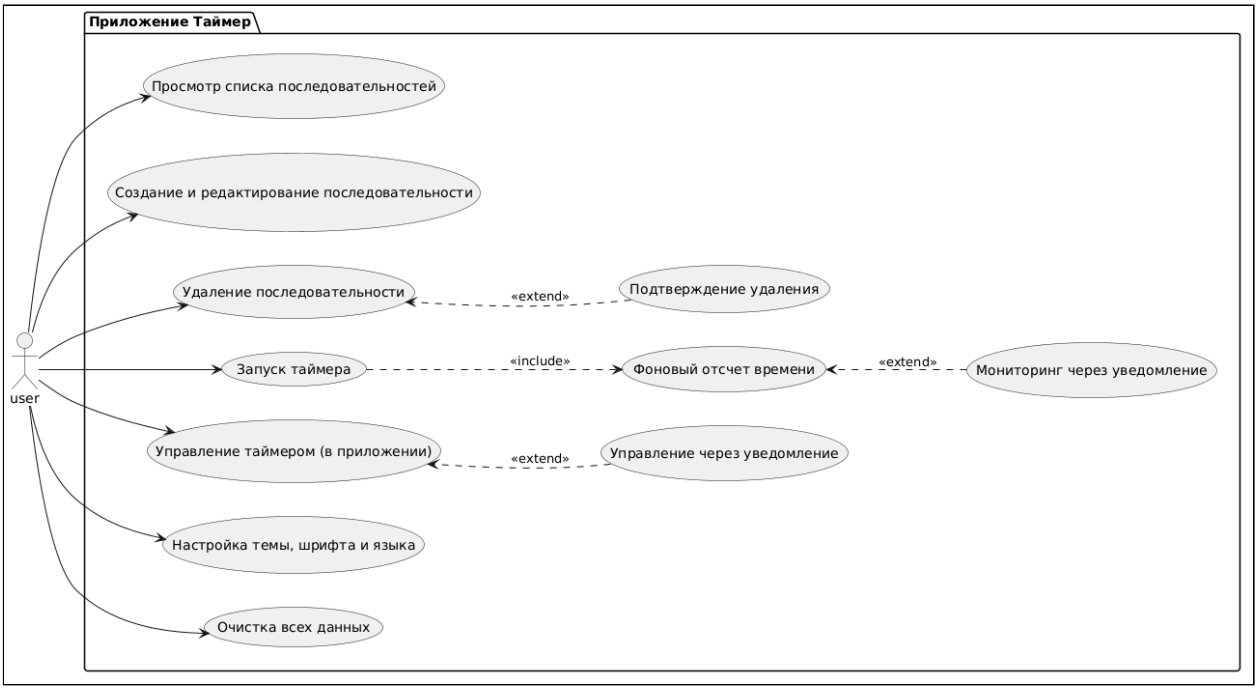

Рисунок 2.1 – Диаграмма вариантов использования

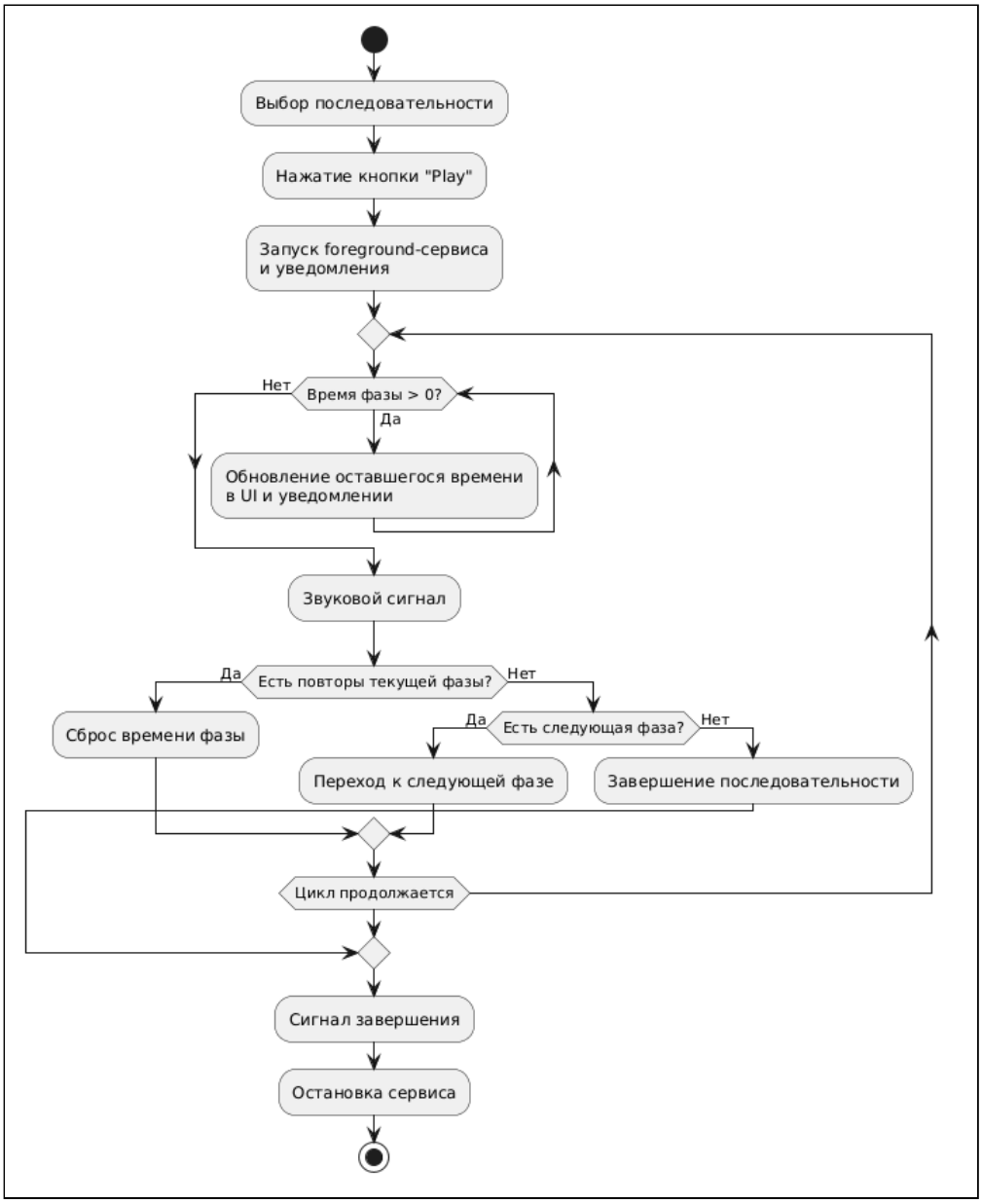

Рисунок 2.2 – Диаграмма активностей выполнения таймера

2.2 Экран списка последовательностей

Главный экран приложения отображает список созданных пользователем последовательностей таймеров в виде карточек. При запуске приложения выполняется загрузка данных из базы данных, после чего список последовательностей становится доступным для взаимодействия. Каждая карточка содержит название последовательности, количество фаз и цветовую метку, позволяющую визуально различать последовательности. Цвет последовательности задаётся пользователем при создании или редактировании и хранится в базе данных в виде значения ARGB.

На каждой карточке расположены три кнопки управления. Для запуска таймера пользователь нажимает кнопку воспроизведения, после чего происходит переход на экран воспроизведения таймера. Редактирование последова-

тельности выполняется нажатием соответствующей кнопки на карточке, что открывает экран редактора с загруженными данными. Удаление последовательности выполняется нажатием кнопки удаления на карточке, после чего отображается диалоговое окно с запросом подтверждения действия. Кнопка создания новой последовательности расположена в правом нижнем углу экрана и открывает экран редактора.

Список последовательностей загружается из базы данных реактивно. Данные автоматически обновляются при любых изменениях в таблице последовательностей. Сортировка выполняется по времени последнего изменения, благодаря чему недавно изменённые последовательности отображаются в верхней части списка. На рисунке 2.3 представлен интерфейс главного экрана приложения.

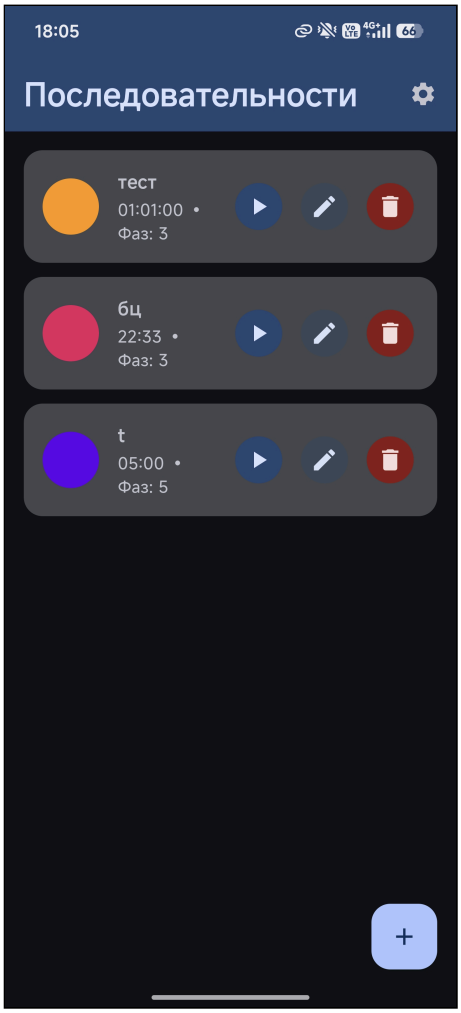

Рисунок 2.3 – Главный экран со списком последовательностей

Приложение поддерживает полный набор операций для работы с последовательностями: создание, чтение, обновление и удаление. Создание новой последовательности выполняется через экран редактора, где пользователь задаёт название, цвет и добавляет фазы. Каждая фаза характеризуется типом (разминка, работа, отдых, заминка), длительностью в секундах и количеством повторений. Обновление последовательности выполняется аналогично созда-

нию, но данные загружаются из базы и отображаются в редакторе. При удалении последовательности автоматически удаляются все связанные с ней фазы.

2.3 Экран таймера

Экран воспроизведения таймера отображает текущее состояние выполнения последовательности и предоставляет элементы управления. В центре экрана расположен круговой индикатор прогресса, визуализирующий оставшееся время текущей фазы. Внутри круга отображается оставшееся время в формате минут и секунд. Над индикатором выводится название текущей фазы, информация о номере текущего повторения и общем количестве повторений фазы.

Ниже основной информации расположен список предстоящих фаз последовательности, позволяющий пользователю видеть последующие этапы выполнения. Каждая предстоящая фаза отображается с указанием типа, длительности и количества повторений. Панель управления в нижней части экрана содержит кнопки для управления воспроизведением: пауза и возобновление, остановка таймера, переход к предыдущей и следующей фазам.

Состояние таймера управляется фоновым сервисом, работающим независимо от Activity. Экран подписывается на поток данных о состоянии и обновляет интерфейс при каждом изменении. Когда пользователь сворачивает приложение, таймер продолжает работать, а уведомление отображает актуальную информацию о выполнении последовательности. На рисунке 2.4 представлен интерфейс экрана воспроизведения таймера.

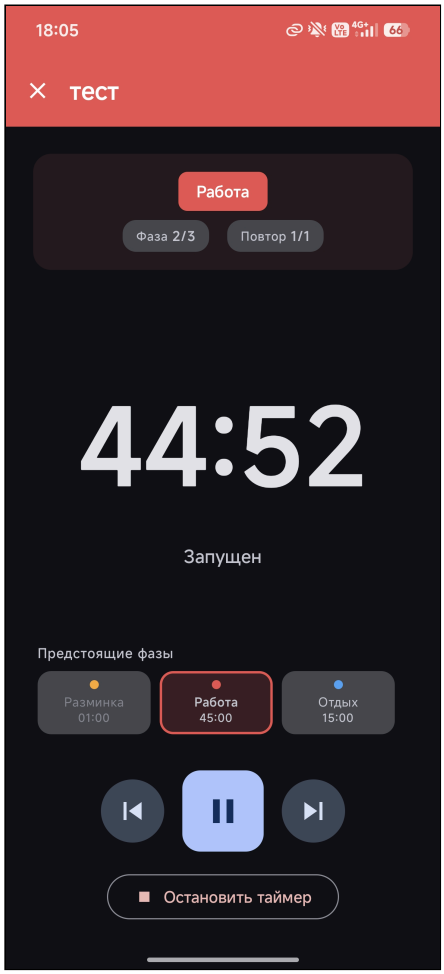

Рисунок 2.4 – Экран воспроизведения таймера

Сервис использует таймер обратного отсчёта для периодического обновления оставшегося времени с интервалом в одну секунду. При завершении фазы воспроизводится звуковой сигнал и принимается решение о дальнейших действиях. Если текущая фаза имеет непройденные повторения, счётчик увеличивается, а время сбрасывается к начальной длительности фазы. Если все повторения завершены, происходит переход к следующей фазе последовательности. При завершении всех фаз сервис публикует состояние завершения последовательности и останавливает фоновую работу.

2.4 Экран настроек

Экран настроек предоставляет пользователю возможность изменения параметров оформления интерфейса и управления данными приложения. Переключатель темы в виде тумблера позволяет выбрать между светлым и тёмным оформлением. Изменение размера шрифта выполняется через список вариантов с радио-кнопками: «Маленький» (85%), «Средний» (100%), «Большой» (115%) и «Очень большой» (130%). Переключение языка интерфейса также представлено в виде списка с радио-кнопками и поддерживает англий-

ский и русский языки. При изменении языка приложение применяет новую локализацию и пересоздаёт Activity для обновления интерфейса.

Также на экране настроек доступны функции удаления всех последовательностей таймеров и очистки настроек с возвратом к значениям по умолчанию. Удаление всех последовательностей удаляет данные из всех связанных таблиц базы данных. Сброс настроек очищает хранилище пользовательских предпочтений, после чего значения параметров возвращаются к заданным по умолчанию.

Все настройки сохраняются в хранилище пользовательских предпочтений, которое предоставляет асинхронный доступ к данным. Изменения настроек отображаются в интерфейсе немедленно благодаря реактивному обновлению состояния. На рисунке 2.5 представлен интерфейс экрана настроек приложения.

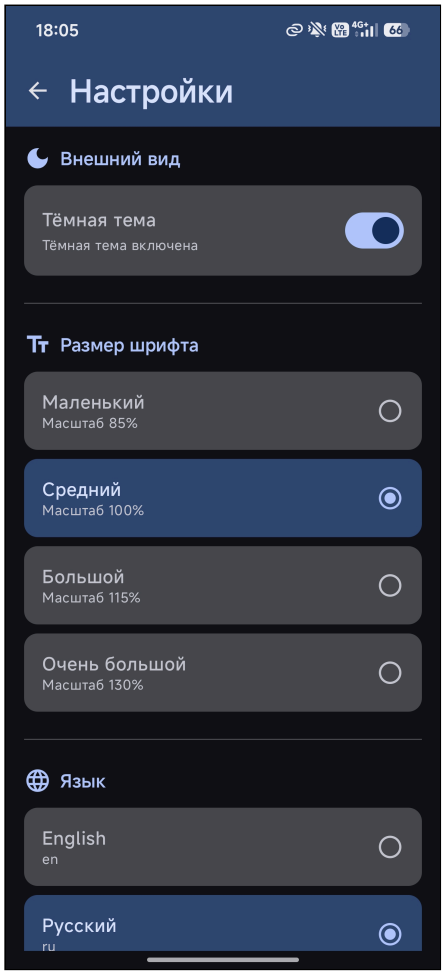

Рисунок 2.5 – Экран настроек приложения

2.5 Уведомление таймера

Для обеспечения работы таймера в фоновом режиме реализован сервис, который отображает постоянное уведомление с информацией о текущем состоянии таймера и кнопками управления. Уведомление отображает название

выполняемой последовательности, название и оставшееся время текущей фазы, номер текущего повторения. Кнопки управления позволяют приостановить и возобновить выполнение, остановить таймер, а также перейти к следующей или предыдущей фазе без открытия приложения. Для устройств Xiaomi уведомление адаптировано для отображения в стиле *Dynamic Island* средствами библиотеки *hyperisland_kit*, что обеспечивает нативный опыт управления в оболочке *HyperOS*. На рисунке 2.6 представлен интерфейс уведомления сервиса.

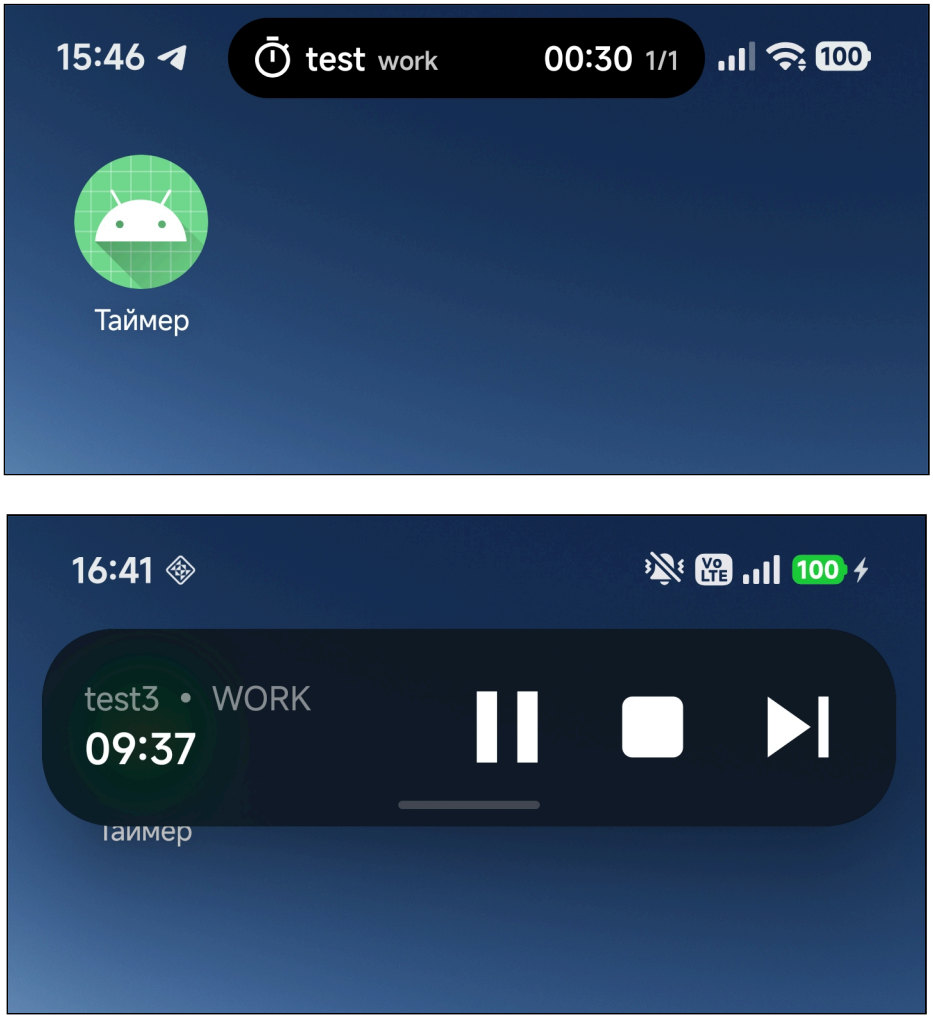

Рисунок 2.6 – Уведомление фонового сервиса

Сервис работает в режиме, предотвращающем его принудительное завершение системой, что обеспечивает непрерывную работу таймера даже при низком уровне заряда батареи или очистке памяти.

ВЫВОД

В ходе выполнения лабораторной работы было разработано мобильное приложение «Таймер» для платформы *Android*. Реализована чёткая архитектура с разделением уровней представления, бизнес-логики и данных. Локальное хранилище на основе базы данных обеспечивает надёжное сохранение последовательностей таймеров и поддержку создания, чтения, обновления и удаления данных. При запуске приложения данные загружаются из базы и отображаются в виде списка, предоставляя пользователю возможность выбора и воспроизведения последовательности. Каждая последовательность хранится вместе с произвольным количеством фаз, описывающих тип активности, длительность и количество повторений.

Фоновый сервис позволяет таймеру работать независимо от интерфейса с отображением уведомлений и управлением через системную панель. Сервис публикует состояние через потоки данных, что обеспечивает автоматическое обновление интерфейса при изменении текущей фазы или оставшегося времени. Система хранения пользовательских настроек обеспечивает сохранение предпочтений темы, размера шрифта и языка интерфейса с поддержкой их немедленного применения. Использование платформенно-специфичных возможностей, таких как *Dynamic Island*, улучшает пользовательский опыт на соответствующих устройствах.

Подготовленные диаграммы наглядно демонстрируют основные сценарии взаимодействия пользователя с системой и подтверждают корректность спроектированной архитектуры. Приложение обеспечивает удобный интерфейс для управления интервальными тренировками и другими многофазными таймерами, сохраняя данные между запусками и работая независимо от состояния Activity.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг программного кода

Листинг А.1 – Определение сущности TimerSequence

```
@Entity(tableName = "timer_sequences")
data class TimerSequence(
    @PrimaryKey(autoGenerate = true)
    val id: Long = 0,

    @ColumnInfo(name = "name")
    val name: String,

    @ColumnInfo(name = "color_argb")
    val colorArgb: Int,

    @ColumnInfo(name = "created_at")
    val createdAt: Long = System.currentTimeMillis(),

    @ColumnInfo(name = "updated_at")
    val updatedAt: Long = System.currentTimeMillis()
)
```

Листинг А.2 – Интерфейс TimerSequenceDao

```
@Dao
interface TimerSequenceDao {

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insert(sequence: TimerSequence): Long

    @Update
    suspend fun update(sequence: TimerSequence)

    @Delete
    suspend fun delete(sequence: TimerSequence)

    @Query("SELECT * FROM timer_sequences WHERE id = :id")
    suspend fun getById(id: Long): TimerSequence?

    @Transaction
    @Query("SELECT * FROM timer_sequences WHERE id = :id")
    suspend fun getSequenceWithPhases(id: Long):
TimerSequenceWithPhases?

    @Transaction
    @Query("SELECT * FROM timer_sequences ORDER BY updated_at DESC")
    fun getAllSequencesWithPhasesFlow():
Flow<List<TimerSequenceWithPhases>>

    @Query("DELETE FROM timer_sequences")
    suspend fun deleteAll()

    @Query("SELECT COUNT(*) FROM timer_sequences")
```

```
    suspend fun getCount(): Int
}
```

Листинг А.3 – Класс PreferencesManager

```
class PreferencesManager private constructor(
    private val context: Context
) {
    private val dataStore: DataStore<Preferences> = context.dataStore

    companion object {
        private val KEY_DARK_THEME =
booleanPreferencesKey("dark_theme")
        private val KEY_FONT_SIZE = intPreferencesKey("font_size")
        private val KEY_LANGUAGE = stringPreferencesKey("language")
    }

    val userPreferencesFlow: Flow<UserPreferences> =
        dataStore.data.map { preferences ->
            UserPreferences(
                isDarkTheme = preferences[KEY_DARK_THEME] ?: false,
                fontSize = FontSize.fromOrdinal(
                    preferences[KEY_FONT_SIZE] ?:
FontSize.MEDIUM.ordinal
                ),
                language = Language.fromCode(
                    preferences[KEY_LANGUAGE] ?: Language.ENGLISH.code
                )
            )
        }

    suspend fun setDarkTheme(isDark: Boolean) {
        dataStore.edit { preferences ->
            preferences[KEY_DARK_THEME] = isDark
        }
    }
}
```

Листинг А.4 – Метод startSequence класса TimerService

```
private fun startSequence(sequenceId: Long) {
    serviceScope.launch {
        try {
            if (_timerState.value.playbackState ==
PlaybackState.RUNNING ||
                _timerState.value.playbackState ==
PlaybackState.PAUSED) {
                stopCountdown()
            }

            val sequence = repository.getSequenceById(sequenceId)

            if (sequence == null || sequence.phases.isEmpty()) {
                Log.e(TAG, "Invalid sequence or no phases found")
                stopTimer()
```

```
                return@launch
            }

            currentSequence = sequence
            currentPhasesList = sequence.phases.sortedBy { it.order }

            currentPhaseIndex = 0
            currentRepetitionIndex = 0

            val firstPhase = currentPhasesList[0]
            currentPhaseRemainingSeconds = firstPhase.durationSeconds

            updateState(
                playbackState = PlaybackState.RUNNING,
                sequenceId = sequence.id,
                sequenceName = sequence.name,
                currentPhaseIndex = currentPhaseIndex,
                currentRepetitionIndex = currentRepetitionIndex,
                totalRepetitions = firstPhase.repetitions,
                currentPhaseType = firstPhase.phaseType,
                totalPhases = currentPhasesList.size,
                allPhases = currentPhasesList,
                remainingSeconds = currentPhaseRemainingSeconds,
                phaseDurationSeconds = firstPhase.durationSeconds
            )

            startForegroundService()

            startCountdown()

        } catch (e: Exception) {
            Log.e(TAG, "Error starting sequence", e)
            stopTimer()
        }
    }
}
```

Листинг А.5 – Обработка завершения фазы в TimerService

```
private fun onPhaseComplete() {
    playBeep()

    val currentPhase =
currentPhasesList.getOrNull(currentPhaseIndex) ?: return

    if (currentRepetitionIndex < currentPhase.repetitions - 1) {
        currentRepetitionIndex++
        currentPhaseRemainingSeconds = currentPhase.durationSeconds

        updateState(
            playbackState = PlaybackState.RUNNING,
            currentRepetitionIndex = currentRepetitionIndex,
            remainingSeconds = currentPhaseRemainingSeconds
        )

        startCountdown()
```

```
    } else {
        if (currentPhaseIndex < currentPhasesList.size - 1) {
            currentPhaseIndex++
            currentRepetitionIndex = 0

            val nextPhase = currentPhasesList[currentPhaseIndex]
            currentPhaseRemainingSeconds = nextPhase.durationSeconds

            updateState(
                playbackState = PlaybackState.RUNNING,
                currentPhaseIndex = currentPhaseIndex,
                currentRepetitionIndex = currentRepetitionIndex,
                totalRepetitions = nextPhase.repetitions,
                currentPhaseType = nextPhase.phaseType,
                remainingSeconds = currentPhaseRemainingSeconds,
                phaseDurationSeconds = nextPhase.durationSeconds
            )

            startCountdown()
        } else {
            onSequenceComplete()
        }
    }
}
```

Листинг А.6 – Создание уведомления для Xiaomi HyperOS

```
private fun buildXiaomiNotification(state: TimerState): Notification {
    val timerIcon = HyperPicture("timer_icon", context,
R.drawable.ic_timer)

    val hyperBuilder = HyperIslandNotification.Builder(
        context = context,
        businessName = "timer_service",
        ticker = CHANNEL_NAME
    )

    hyperBuilder.addPicture(timerIcon)

    val actionKeys = addIconActions(hyperBuilder, state)

    hyperBuilder.setBaseInfo(
        title = state.getFormattedRemainingTime(),
        content = "${getNotificationTitle(state)} •
${state.currentPhaseType.getLocalizedName(context)}",
        actionKeys = actionKeys
    )

    hyperBuilder.setIslandConfig(
        priority = 0,
        dismissible = false,
    )

    hyperBuilder.setSmallIsland("timer_icon")
    hyperBuilder.setIslandFirstFloat(false)
    hyperBuilder.setEnableFloat(false)
```

```
    hyperBuilder.setBigIslandInfo(
        left = ImageTextInfoLeft(
            type = 1,
            picInfo = PicInfo(type = 1, pic = "timer_icon"),
            textInfo = TextInfo(
                title = state.sequenceName + " ",
                content =
state.currentPhaseType.getLocalizedName(context).lowercase(getDefault())
            )
        ),
        right = ImageTextInfoRight(
            type = 2,
            textInfo = TextInfo(
                title = state.getFormattedRemainingTime() + " ",
                content = state.getRepetitionDisplay()
            )
        )
    )

    val resourceBundle = hyperBuilder.buildResourceBundle()
    val jsonParam = hyperBuilder.buildJsonParam()

    return NotificationCompat.Builder(context, CHANNEL_ID)
        .setSmallIcon(R.drawable.ic_timer)
        .setContentTitle(getNotificationTitle(state))
        .setContentText(getNotificationContent(state))
        .setOngoing(state.playbackState == PlaybackState.RUNNING)
        .setContentIntent(createContentIntent(state))
        .setPriority(NotificationCompat.PRIORITY_LOW)
        .setVisibility(NotificationCompat.VISIBILITY_PUBLIC)
        .addExtras(resourceBundle)
        .setExtras(resourceBundle.apply {
            putString("miui.focus.param", jsonParam)
        })
        .build()
}
```